

Midas F405 CAN 通信协议说明

基于当前工程代码行为整理 - 生成日期 2026-08-06

项目	F:/Midas/Midas_F405 当前工作区
协议类型	CAN 2.0A 标准帧, 11-bit 标识符, 数据帧
发送 ID 组帧	StdId = (node_id << 5) cmd
数据编码	除 MIT 指令外, 浮点数据均为 little-endian IEEE754 float32
当前代码默认波特率枚举	0 = 1M, 1 = 500K, 2 = 250K, 3 = 125K
核心源文件	Hardware/can.h, Hardware/can.c, Communication/communication_can.c, User/Src/main.c

1. 总体规则

- 协议使用标准帧, 不使用扩展帧。发送函数 OdriveSendData() 直接把 11-bit StdId 组织为 (node_id << 5) | cmd。
- 低 5 bit 为命令码 cmd, 高位为节点号 node_id。当前命令枚举主要分布在 0x000 - 0x01F。
- 常规 4 字节或 8 字节数据按 little-endian float32 编码。例如一个 float 会按低字节在前写入 Data[0..3]。
- 接收过滤器当前配置为全接收, 软件层再做 StdId 解析。
- 位置单位在代码中主要使用 turn, 速度单位为 turn/s, Iq 和母线电流单位为 A, 母线电压单位为 V。

2. 标识符与节点号说明

字段	位宽	当前代码含义	说明
node_id	高 6 bit 逻辑字段	StdId >> 5	发送时由 OD_CANID 参与组帧
cmd	低 5 bit	StdId & 0x1F	对应 MSG_xxx 命令码
重要注意	-	当前 RX 代码只进入 case AXIS0_ID	按现状只有 node_id = 0 的命令会被实际处理。OD_CANID 可用于发送, 但 RX 分发没有完全按 OD_CANID 适配。

3. ID 计算示例

- 示例 1: node_id = 0, cmd = 0x00D, 则 StdId = 0x00D。
- 示例 2: node_id = 3, cmd = 0x009, 则 StdId = (3 << 5) + 0x009 = 0x069。
- 注意: 虽然发送侧支持任意 node_id 组帧, 但当前接收分发只真正处理 node_id = 0。

4. 已实现的接收命令 - 第 1 页

命令名	cmd	DLC	载荷格式	当前代码行为	响应
MSG_SET_AXIS_REQUESTED_STATE	0x007	>= 1	Data[0] = 轴状态枚举	直接写 current_state_ = Data[0]。状态值见后文状态表。	无立即回包
MSG_GET_ENCODER_ESTIMATES	0x009	任意	请求载荷未使用	收到后立即发送同 cmd 回包。	回包 8B: pos_estimate_, vel_estimate_
MSG_SET_CONTROLLER_MODES	0x00B	>= 1	Data[0] = 模式选择值 1..8	设置 control_mode 与 input_mode 组合。	无
MSG_SET_INPUT_POS	0x00C	4	float32 目标位置	写 input_pos_，并置 input_pos_updated_ = true。	无
MSG_SET_INPUT_VEL	0x00D	4	float32 目标速度	写 input_vel_。	无
MSG_SET_INPUT_TORQUE	0x00E	4	float32 扭矩目标	写 input_torque_。辅助函数名中有 CUR 字样，但实际变量是 torque 输入。	无
MSG_SET_LIMITS	0x00F	8	float32 current_lim + float32 vel_limit	更新 motor_config.current_lim 与 ctrl_config.vel_limit。	无
MSG_SET_TRAJ_VEL_LIMIT	0x011	4	float32 轨迹速度限制	更新 trapTraj_config.vel_limit。	无

4. 已实现的接收命令 - 第 2 页

命令名	cmd	DLC	载荷格式	当前代码行为	响应
MSG_SET_TRAJ_ACCEL_LIMITS	0x012	8	float32 accel_limit + float32 decel_limit	更新 trapTraj_config.accel_limit / decel_limit。	无
MSG_GET_IQ	0x014	任意	请求载荷未使用	收到后立即发送同 cmd 回包。	回包 8B: Idq_setpoint_q, Iq_measured
MSG_SET_MIT_CONTROL	0x015	8	MIT 打包格式	解析位置、速度、kp、kd、扭矩并写入 MIT 目标量。	无
MSG_CLEAR_ERRORS	0x017	任意	载荷未使用	仅执行 motor_error = 0。	无
MSG_SET_POS_GAIN	0x019	4	float32 pos_gain	更新 ctrl_config.pos_gain。	无
MSG_SET_VEL_GAINS	0x01A	8	float32 vel_gain + float32 vel_integrator_gain	更新 ctrl_config.vel_gain 与 ctrl_config.vel_integrator_gain。	无
MSG_RESET_ODRIVE	0x01B	任意	载荷未使用	调用 NVIC_SystemReset()。	设备复位
MSG_SAVE_CONFIG	0x01C	任意	载荷未使用	调用 flash_para_write() 保存参数。	无

5. 已定义但当前未真正实现的命令

命令名	cmd	当前状态
MSG_CO_NMT_CTRL	0x000	进入 switch, 但当前未处理
MSG_ODRIVE_HEARTBEAT	0x001	预留, 当前未处理
MSG_ODRIVE_ESTOP	0x002	预留, 当前未处理
MSG_GET_MOTOR_ERROR	0x003	枚举存在, 但当前无回包实现
MSG_GET_ENCODER_ERROR	0x004	枚举存在, 但当前无回包实现
MSG_GET_SENSORLESS_ERROR	0x005	枚举存在, 但当前无回包实现
MSG_SET_AXIS_NODE_ID	0x006	枚举存在, 但当前未实现运行时改 node_id
MSG_SET_AXIS_STARTUP_CONFIG	0x008	枚举存在, 但当前未处理
MSG_GET_ENCODER_COUNT	0x00A	枚举存在, 但当前未回传编码器计数
MSG_START_ANTICOGGING	0x010	枚举存在, 但当前未处理
MSG_SET_TRAJ_INERTIA	0x013	枚举存在, 但当前未处理
MSG_GET_BUS_VOLTAGE_CURRENT	0x016	当前只有周期上报, 没有按命令请求回包
MSG_SET_LINEAR_COUNT	0x018	枚举存在, 但当前未处理
MSG_GET_MOTOR_TEMP	0x01D	发送代码被注释, 当前无实际通路
MSG_GET_POS_GAIN	0x01E	枚举存在, 但当前无回包
MSG_GET_VEL_GAINS	0x01F	枚举存在, 但当前无回包

6. 控制模式映射 - MSG_SET_CONTROLLER_MODES

Data[0]	control_mode	input_mode	说明
1	POSITION_CONTROL	TRAP_TRAJ	位置控制 - 梯形轨迹
2	POSITION_CONTROL	POS_FILTER	位置控制 - 位置滤波
3	POSITION_CONTROL	PASSTHROUGH	位置控制 - 位置直通
4	VELOCITY_CONTROL	VEL_RAMP	速度控制 - 速度斜坡
5	VELOCITY_CONTROL	PASSTHROUGH	速度控制 - 速度直通
6	TORQUE_CONTROL	TORQUE_RAMP	扭矩控制 - 扭矩斜坡
7	TORQUE_CONTROL	PASSTHROUGH	扭矩控制 - 扭矩直通
8	TORQUE_CONTROL	MIT	MIT 模式
其它	保持/切到无效输入	INACTIVE	default 分支把 input_mode 置为 INACTIVE

7. 轴状态枚举 - MSG_SET_AXIS_REQUESTED_STATE

值	状态名	说明
0	AXIS_STATE_IDLE	空闲
1	AXIS_STATE_MOTOR_CALIBRATION	电机参数校准
2	AXIS_STATE_ENCODER_INDEX_SEARCH	编码器索引搜索
3	AXIS_STATE_ENCODER_DIR_FIND	编码器方向查找
4	AXIS_STATE_ENCODER_HALL_POLARITY_CALIBRATION	霍尔极性校准
5	AXIS_STATE_ENCODER_HALL_PHASE_CALIBRATION	霍尔相位校准
6	AXIS_STATE_ENCODER_OFFSET_CALIBRATION	编码器偏移校准
7	AXIS_STATE_LOCKIN_SPIN	锁相旋转
8	AXIS_STATE_CLOSED_LOOP_CONTROL	闭环控制
9	AXIS_STATE_UNDEFINED	未定义

8. MIT 模式 8 字节打包格式

- MIT 指令不是 float32，而是自定义位打包。解析顺序为 p(16bit), v(12bit), kp(12bit), kd(12bit), t(12bit)。
- 解包量程由当前代码硬编码：position [-20000, 20000]，velocity [-2000, 2000]，kp [-200, 200]，kd [-200, 200]，torque [-20, 20]。

字段	位宽	位布局	量程
p	16 bit	Data[0] 高 8 bit + Data[1] 低 8 bit	[-20000, 20000]
v	12 bit	Data[2] 全 8 bit + Data[3] 高 4 bit	[-2000, 2000]
kp	12 bit	Data[3] 低 4 bit + Data[4] 全 8 bit	[-200, 200]
kd	12 bit	Data[5] 全 8 bit + Data[6] 高 4 bit	[-200, 200]
t	12 bit	Data[6] 低 4 bit + Data[7] 全 8 bit	[-20, 20]

9. 周期上报帧

cmd	上报内容	载荷	理论周期	说明
0x009	编码器估计	pos_estimate_, vel_estimate_	理论 100 Hz	Motor_CAN_Send_Data() 中每 can_cnt % 10 == 0 发送
0x014	lq 目标与实测	ldq_setpoint_q, lq_measured	理论 100 Hz	与编码器估计同批发送
0x016	母线电压与电流	vbus_voltage, lbus	理论 100 Hz	当前仅周期上报，没有单独命令请求回包

- can_cnt 在 TIM7_IRQHandler() 中递增，因此只有 TIM7 中断实际开启时，以上周期上报才会工作。
- 当前 User/Src/main.c 中 TIM7_Init() 已被注释。若没有其它地方启动 TIM7，则周期上报通路不会生效。
- 温度上报 MSG_GET_MOTOR_TEMP 的发送代码目前处于注释状态。

10. 特殊帧 - 机致科技中空编码器回传

当收到 StdId == 0x64 的帧时，代码会走单独解析逻辑 En_d40_read()。该逻辑不使用 (node_id << 5) | cmd 规则，而是从 Data[6]、Data[7]、Data[1]、Data[2] 组合出原始值，再截取 20 bit 单圈绝对位置并换算角度。该帧属于当前工程的辅助解析逻辑，不属于主 CAN 控制协议。

11. 对接建议与当前代码注意事项

- 如果上位机要稳定控制当前固件，命令节点号建议先按 node_id = 0 使用，因为 RX 分发当前只真正处理 node_id = 0。
- 如需多节点或可配置节点号，建议把 RX 分发从 case AXIS0_ID 改为与 OD_CANID 一致的比较逻辑。
- GET 类命令并不是都能请求回包，当前真正实现按请求返回的只有 0x009 和 0x014。
- MSG_CLEAR_ERRORS 当前只清 motor_error，没有扩展到所有子模块错误字。
- 本文档仅覆盖当前代码中的 CAN 通道实现，不包含 USB、USART、VOFA 调试口协议。